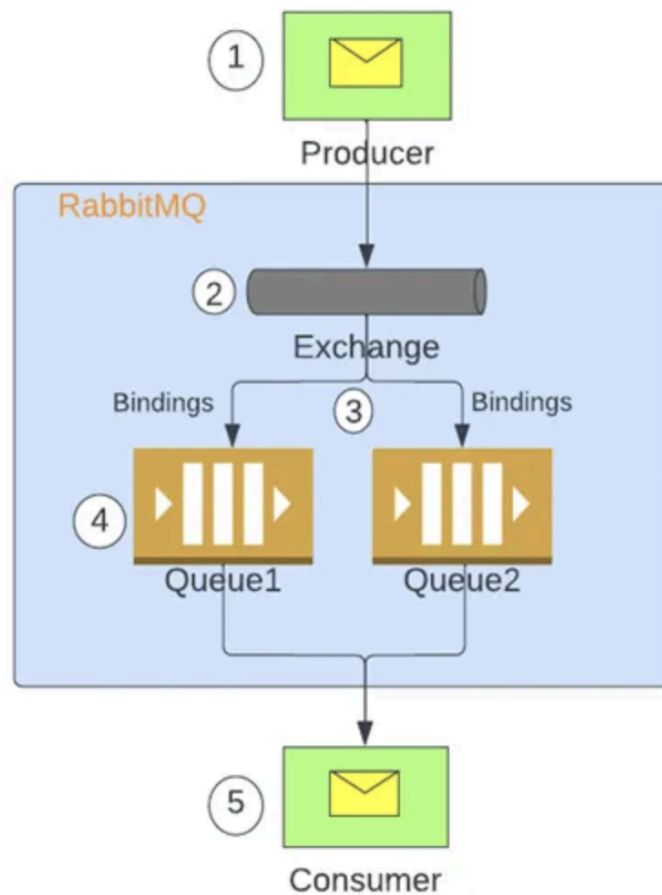


RMQ Assignment

Message Flow in RabbitMQ



RabbitMQ is an open-source message broker that primarily implements the Advanced Message Queuing Protocol (AMQP). Its fundamental purpose is to **decouple** applications by managing the reliable, asynchronous flow of messages between **Producers** (senders) and **Consumers** (receivers).

Handling High Load

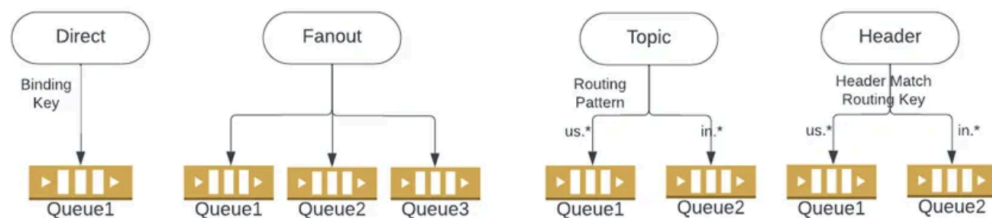
Handling Failures

Improved User Experience

System Resilience and Scalability

The process of routing a message is precise and involves a few steps :

1. **Publishing:** The **Producer** sends a message to a specific **Exchange**. When the Exchange is created, its routing behavior (**Type**) must be specified.
2. **Routing Decision:** The Exchange receives the message and consults its rules, taking into account various message attributes (like the **Routing Key**), depending on its type.
3. **Binding Check: Bindings**—which are formal links between the Exchange and various Queues—are used to determine the destinations. The Exchange routes the message into all matching Queues based on these Bindings. Present on queue
4. **Queue Storage:** Messages remain stored safely in the **Queue** until a Consumer connects and retrieves them.
5. **Consumption:** The **Consumer** handles the message, performing the required task.



1. Install and configure 1 node RMQ cluster version 3.8.9

Step 1.1 : ***multipass launch --name rmq-node-1 20.04***

Step 1.2 : ***multipass shell rmq-node-1***

Step 1.3 : ***sudo apt-get update -y***

Step 1.4 : ***sudo apt-get install -y erlang-base erlang-crypto erlang-ssl erlang-public-key erlang-mnesia erlang-tools erlang-os-mon erlang-runtime-tools***

Installs the core Erlang runtime environment components required for RabbitMQ to function. Because RMQ runs on erlang virtual machine. These specific packages are "sub-modules" of Erlang

Step 1.5 : **wget**

https://github.com/rabbitmq/rabbitmq-server/releases/download/v3.8.9/rabbitmq-server_3.8.9-1_all.deb

Download the specific version 3.8.9 RabbitMQ installer package from GitHub to our machine.

Step 1.6 : **sudo apt --fix-broken install -y**

**sudo apt-get install -y **

**erlang-base **

**erlang-crypto **

**erlang-ssl **

**erlang-public-key **

**erlang-mnesia **

**erlang-tools **

**erlang-os-mon **

**erlang-runtime-tools **

**erlang-eldap **

**erlang-inets **

**erlang-parsetools **

**erlang-xmerl **

socat

Automatically detects and installs any missing dependencies that caused the previous **dpkg** command to fail.

Step 1.7 : **sudo dpkg -i rabbitmq-server_3.8.9-1_all.deb**

Attempts to install the RabbitMQ package manually using the Debian package manager.

Step 1.8 : **sudo systemctl unmask rabbitmq-server.service**

Removes a "mask" on the service, ensuring it is allowed to be started by the system.

Step 1.9 : **sudo systemctl enable rabbitmq-server**

Start the service

Step 1.10 : **sudo systemctl start rabbitmq-server**

Verify the status

Step 1.11 : **sudo systemctl status rabbitmq-server**

```

ubuntu@rmq-node-1:~$ sudo systemctl restart rabbitmq-server
ubuntu@rmq-node-1:~$ sudo systemctl status rabbitmq-server
● rabbitmq-server.service - RabbitMQ broker
   Loaded: loaded (/lib/systemd/system/rabbitmq-server.service; enabled; vendor preset: enabled)
   Active: active (running) since Thu 2025-12-11 16:23:03 IST; 4s ago
     Main PID: 40722 (beam.smp)
        Status: "Initialized"
          Tasks: 84 (limit: 1084)
         Memory: 131.6M
       CGroup: /system.slice/rabbitmq-server.service
               └─40722 /usr/lib/erlang/erts-10.6.4/bin/beam.smp -W w -K true -A 64 -MBas ageffcbf -MHas ageffcbf -MBlmbcs 512 -MHlmbcs 512 -MMmcs 30 -P 1048576 -t 5000000 -stbt db -zdbbl 128000 -- -root /usr/lib/erl
               └─40830 erl_child_setup 32768
               └─40875 inet_gethost 4
               └─40876 inet_gethost 4

Dec 11 16:23:02 rmq-node-1 rabbitmq-server[40722]: ##### Licensed under the MPL 2.0. Website: https://rabbitmq.com
Dec 11 16:23:02 rmq-node-1 rabbitmq-server[40722]: Doc guides: https://rabbitmq.com/documentation.html
Dec 11 16:23:02 rmq-node-1 rabbitmq-server[40722]: Support: https://rabbitmq.com/contact.html
Dec 11 16:23:02 rmq-node-1 rabbitmq-server[40722]: Tutorials: https://rabbitmq.com/getstarted.html
Dec 11 16:23:02 rmq-node-1 rabbitmq-server[40722]: Monitoring: https://rabbitmq.com/monitoring.html
Dec 11 16:23:02 rmq-node-1 rabbitmq-server[40722]: Logs: /var/log/rabbitmq/rabbit@rmq-node-1.log
Dec 11 16:23:02 rmq-node-1 rabbitmq-server[40722]: /var/log/rabbitmq/rabbit@rmq-node-1_upgrade.log
Dec 11 16:23:02 rmq-node-1 rabbitmq-server[40722]: Config file(s): /etc/rabbitmq/advanced.config
Dec 11 16:23:03 rmq-node-1 rabbitmq-server[40722]: Starting broker... completed with 3 plugins.
Dec 11 16:23:03 rmq-node-1 systemd[1]: Started RabbitMQ broker.
lines 1-23/23 (END)
^C

```

2. The RMQ cluster should be on TLS

Step 2.1 : **sudo mkdir -p /etc/rabbitmq/certs**

Creates a dedicated directory to store your security certificates and private keys.

Step 2.2 : **cd /tmp**

Step 2.3 : **VM_HOST=\$(hostname -f)**

Saves the fully qualified domain name of your VM into a variable for use in the certificate.

Step 2.4 : **VM_IP="192.168.64.56"**

Step 2.5 : **sudo tee openssl-rmq.cnf <<EOF**

```

[ req ]
distinguished_name = req_distinguished_name
prompt = no

[ req_distinguished_name ]
CN = $VM_HOST
O = RabbitMQTestOrg

[ v3_req ]
basicConstraints = CA:FALSE
keyUsage = digitalSignature, keyEncipherment, dataEncipherment, keyCertSign
extendedKeyUsage = serverAuth, clientAuth
subjectAltName = @alt_names

[ alt_names ]
DNS.1 = $VM_HOST
IP.1 = $VM_IP
EOF

```

Creates a configuration file that tells OpenSSL exactly how to format the certificate (DNS, IP, etc.). Its like filling an application form to build an id card for our system. Through this file we will create a private key and a certificate

Step 2.6 : ***sudo openssl genrsa -out server_key.pem 2048***

Generates a 2048-bit RSA private key for the RabbitMQ server.

Step 2.7 : ***sudo openssl req -new -key server_key.pem -out server_csr.pem -config openssl-rmq.cnf***

Creates a Certificate Signing Request (CSR) using the private key and the config file from Step 2.5.

Step 2.8 : ***sudo openssl x509 -req -days 365 -in server_csr.pem ***
***-CA ca_certificate.pem ***
***-CAkey ca_key.pem ***
***-CAcreateserial -out server_certificate.pem ***
-extfile openssl-rmq.cnf -extensions v3_req

Sign the CSR using your Certificate Authority (CA) to create the final valid server certificate.

Step 2.9 : ***sudo mv ca_certificate.pem ca_key.pem server_certificate.pem server_key.pem /etc/rabbitmq/certs/***

Moves all the generated certificate files to the final RabbitMQ security directory.

Step 2.10 : ***sudo chown -R rabbitmq:rabbitmq /etc/rabbitmq/certs***

Changes ownership of the certificates to the `rabbitmq` user so the service can read them.

Step 2.11 : ***sudo chmod 400 /etc/rabbitmq/certs/****

Restricts file permissions so only the owner can read the certificates, securing the private keys.

Step 2.12 : ***sudo rabbitmq-plugins enable rabbitmq_management***

Activates the web-based management interface and the background API.

When a client connects to RabbitMQ, the **server presents its certificate** as a digital ID. The client uses its copy of the CA certificate to **verify** that this ID is authentic. Once verified, the client sends a secret key to the server, which only the **server can decrypt using its private key**. This process establishes a secure tunnel where all subsequent data is encrypted, ensuring that only the intended RabbitMQ server can read the messages.

Step 2.13 : ***sudo systemctl restart rabbitmq-server***

Step 2.14 : ***sudo systemctl status rabbitmq-server***

Step 2.15 : ***sudo tail /var/log/rabbitmq/rabbit@rmq-node-1.log***

Read the end of the log file to check for any errors related to SSL or plugin startup.

```
ubuntu@rmq-node-1:/tmp$ sudo systemctl status rabbitmq-server
● rabbitmq-server.service - RabbitMQ broker
   Loaded: loaded (/lib/systemd/system/rabbitmq-server.service; enabled; vendor preset: enabled)
   Active: active (running) since Thu 2025-12-11 16:23:03 IST; 4s ago
 Main PID: 40722 (beam.smp)
   Status: "Initialized"
    Tasks: 84 (limit: 1084)
   Memory: 131.6M
   CGroup: /system.slice/rabbitmq-server.service
           └─40722 /usr/lib/erlang/erts-10.6.4/bin/beam.smp -W w -K true -A 64 -MBas ageffcbf -Mlas ageffcbf -Mlmbcs 512 -Mhlmbcs 512 -MMcs 30 -P 1048576 -t 5000000 -stbt db -zdbbl 128000 -- -root /usr/lib/erl
           └─40830 erl_child_setup 32768
           └─40875 inet_gethost 4
           └─40876 inet_gethost 4

Dec 11 16:23:02 rmq-node-1 rabbitmq-server[40722]: ##### Licensed under the MPL 2.0. Website: https://rabbitmq.com
Dec 11 16:23:02 rmq-node-1 rabbitmq-server[40722]: Doc guides: https://rabbitmq.com/documentation.html
Dec 11 16:23:02 rmq-node-1 rabbitmq-server[40722]: Support: https://rabbitmq.com/contact.html
Dec 11 16:23:02 rmq-node-1 rabbitmq-server[40722]: Tutorials: https://rabbitmq.com/getstarted.html
Dec 11 16:23:02 rmq-node-1 rabbitmq-server[40722]: Monitoring: https://rabbitmq.com/monitoring.html
Dec 11 16:23:02 rmq-node-1 rabbitmq-server[40722]: Logs: /var/log/rabbitmq/rabbit@rmq-node-1.log
Dec 11 16:23:02 rmq-node-1 rabbitmq-server[40722]: /var/log/rabbitmq/rabbit@rmq-node-1_upgrade.log
Dec 11 16:23:02 rmq-node-1 rabbitmq-server[40722]: Config file(s): /etc/rabbitmq/advanced.config
Dec 11 16:23:03 rmq-node-1 rabbitmq-server[40722]: Starting broker... completed with 3 plugins.
Dec 11 16:23:03 rmq-node-1 systemd[1]: Started RabbitMQ broker.
lines 1-23/23 (END)
^C
ubuntu@rmq-node-1:/tmp$ sudo tail /var/log/rabbitmq/rabbit@rmq-node-1.log
2025-12-11 16:23:03.300 [info] <0.673.0> Starting worker pool 'management_worker_pool' with 3 processes in it
2025-12-11 16:23:03.301 [info] <0.499.0> Ready to start client connection listeners
2025-12-11 16:23:03.301 [info] <0.43.0> Application rabbitmq_management started on node 'rabbit@rmq-node-1'
2025-12-11 16:23:03.301 [info] <0.695.0> started TCP listener on [::]:5672
2025-12-11 16:23:03.301 [notice] <0.276.0> Changed loghmm of /var/log/rabbitmq/rabbit@rmq-node-1.log to 5000
2025-12-11 16:23:03.400 [info] <0.499.0> Server startup complete; 3 plugins started.
* rabbitmq_management
* rabbitmq_web_dispatch
* rabbitmq_management_agent
2025-12-11 16:23:03.400 [info] <0.499.0> Resetting node maintenance status
ubuntu@rmq-node-1:/tmp$

-
ubuntu@rmq-node-1:/tmp$ sudo tail /var/log/rabbitmq/rabbit@rmq-node-1.log
2025-12-12 10:10:25.564 [info] <0.674.0> Starting worker pool 'management_worker_pool' with 3 processes in it
2025-12-12 10:10:25.564 [info] <0.499.0> Ready to start client connection listeners
2025-12-12 10:10:25.564 [info] <0.43.0> Application rabbitmq_management started on node 'rabbit@rmq-node-1'
2025-12-12 10:10:25.566 [info] <0.697.0> started TLS (SSL) listener on [::]:5671
2025-12-12 10:10:25.566 [notice] <0.276.0> Changed loghmm of /var/log/rabbitmq/rabbit@rmq-node-1.log to 5000
2025-12-12 10:10:25.677 [info] <0.499.0> Server startup complete; 3 plugins started.
* rabbitmq_management
* rabbitmq_web_dispatch
* rabbitmq_management_agent
2025-12-12 10:10:25.678 [info] <0.499.0> Resetting node maintenance status
ubuntu@rmq-node-1:/tmp$
```

3. Vhost, User, and Permissions

Step 3.1 :***NEW_USER="publisher_user"***

Step 3.2 :***NEW_PASS="RMQPassword@1101!"***

Step 3.3 :***VHOST_NAME="/data_processing"***

Defines a Virtual Host (vhost) name, which acts like a private "mini-server" inside RabbitMQ.

Step 3.4 :***sudo rabbitmqctl add_vhost \$VHOST_NAME***

Creates the isolated logical environment (vhost) to keep data separate from other projects.

Step 3.5 :**`sudo rabbitmqctl add_user $NEW_USER $NEW_PASS`**

Creates the new credentials in the RabbitMQ internal database.

Step 3.6 :**`sudo rabbitmqctl set_user_tags $NEW_USER management`**

Grants the user permission to log into the web-based Management Dashboard.

Set Permissions

Step 3.7 :**`sudo rabbitmqctl set_permissions -p $VHOST_NAME $NEW_USER ".*" ".*" ".*"`**

Grants the user full "Configure, Write, and Read" rights only within the specific vhost

Step 3.8 :**`sudo rabbitmqctl delete_user guest`**

Removes the default "guest/guest" account to prevent unauthorized access (a critical security step).

Step 3.9 :

Get VM IP: Find the IPv4 address from your host machine: `multipass info rmq-node-1`

Access Web UI (Secure): Open your browser to `https://<YOUR_VM_IP>:15671` (You will have to accept the security warning for the self-signed certificate).

When you opened the link, you would have seen a browser warning (e.g., "Your connection is not private" or "Potential Security Risk"). This is **expected** because the certificate you created is **self-signed** and is not trusted by public certificate authorities (CAs) like Google or Mozilla.

Action: You must click through the advanced options to proceed/accept the risk and load the page.

Log In: Use your new user: **Username:** `publisher_user` / **Password:** `RMQPassword@1101!`




RabbitMQ 3.8.9 Erlang 22.2.7
Refreshed 2025-12-15 11:29:03 Refresh every 5 seconds
Virtual host All
Cluster rabbit@rmq-node-1
User publisher_user Log out

Overview
Connections
Channels
Exchanges
Queues
Admin

Overview

Totals

Queued messages last minute

Currently idle

Message rates last minute

Currently idle

Global counts

Connections: 0 Channels: 0 Exchanges: 7 Queues: 0

[HTTP API](#)
[Server Docs](#)
[Tutorials](#)
[Community Support](#)
[Community Slack](#)
[Commercial Support](#)
[Plugins](#)
[GitHub](#)
[Changelog](#)

Step 3.10 :Create 2 queues (DATA, DATA_SIDELINE) on the above created vhost

1. Navigate to the **Queues** tab in the management interface.
2. Ensure the **Virtual Host** dropdown is set to **/data_processing**.
3. **Create Queue 1:**
 - **Name:** DATA
 - **Check the Durable box (Crucial for persistence)**
 - Click **Add queue**.
4. **Create Queue 2:**
 - **Name:** DATA_SIDELINE
 - **Check the Durable box (Crucial for persistence)**
 - Click **Add queue**.


RabbitMQ 3.8.9 Erlang 22.2.7
Refreshed 2025-12-15 11:32:44 Refresh every 5 seconds
Virtual host All
Cluster rabbit@rmq-node-1
User publisher_user Log out

Overview
Connections
Channels
Exchanges
Queues
Admin

Queues

All queues (2)

Pagination

Page 1 of 1 Filter: ☐ Regex

Displaying 2 items , page size up to: 100

Overview				Messages			Message rates			
Name	Type	Features	State	Ready	Unacked	Total	incoming	deliver	/ get	ack
DATA	classic	D Args	idle	0	0	0				
DATA_SIDELINE	classic	D Args	idle	0	0	0				

Add a new queue

Type: Classic

Name:

Durability: Durable

Auto delete: No

Arguments:

Add Message TTL Auto expire Max length Max length bytes Overflow behaviour Dead letter exchange Dead letter routing key Single active consumer Maximum priority Lazy mode Master locator

Add queue

[HTTP API](#)
[Server Docs](#)
[Tutorials](#)
[Community Support](#)
[Community Slack](#)
[Commercial Support](#)
[Plugins](#)
[GitHub](#)
[Changelog](#)

4. Testing : Create a publisher and consumer for the DATA queue

Publish 100 messages to DATA queue

In production, our application and our RabbitMQ server live on different machines. Running the code from our host machine proves they can communicate over a network, through firewalls, and across different IP addresses. Since we set up encryption, we need to prove that an **external** client can use the `ca_certificate.pem` to verify the server's identity. If we run it inside the VM, we aren't truly testing if our security works for outside users. It's easier to write, debug, and manage our code using our local tools and IDEs (like VS Code) on our laptop, while letting the VM handle the "heavy lifting" of the RabbitMQ service.

Step 4.1 : ***multipass shell rmq-node-1***

Step 4.2 : ***sudo cp /etc/rabbitmq/certs/ca_certificate.pem /home/ubuntu/***

Copies the restricted CA certificate from the secure RabbitMQ folder to a public area where the 'ubuntu' user has access.

Step 4.3 : ***sudo chown ubuntu:ubuntu /home/ubuntu/ca_certificate.pem***

Changes the file's owner so the standard 'ubuntu' user (and Multipass) has permission to read and move it.

Step 4.4 : ***exit***

Step 4.5 : ***multipass transfer rmq-node-1:/home/ubuntu/ca_certificate.pem ~/***

Downloads the certificate from the VM to your local home directory so your local apps can trust the server.

Step 4.6 : ***ls -l ~/ca_certificate.pem - check to see if file is present***

Lists the file details locally to confirm the transfer was successful and the file exists.

Step 4.7 : ***pip3 install pika***

Install the official RabbitMQ Python library (**Pika**) on your **host machine**.

Step 4.8 : ***vim [publisher.py](#)***

Python

```
import pika # The main library that lets Python talk to RabbitMQ.
import ssl # The library that handles the "locked envelope" (security/TLS).
import time

# --- CONFIGURATION ---
HOST = '192.168.64.56'
PORT = 5671 # Secure TLS Port
VHOST = '/data_processing'
USERNAME = 'publisher_user'
PASSWORD = 'RMQPassword@1101!'

# --- ABSOLUTE PATH CONFIRMED FROM YOUR HOST MACHINE ---
CA_CERT_PATH = '/Users/shambhavi.intern/ca_certificate.pem'
# --- TLS/SSL SETUP (The Safest Compatibility Fix) ---

# 1. Create the SSL Context, explicitly setting the protocol first (avoids
AttributeError)
ssl_context = ssl.SSLContext(ssl.PROTOCOL_TLSv1_2) # defines protocol used to
scramble our data

# 2. Load the CA file to verify the server's certificate
ssl_context.load_verify_locations(cafile=CA_CERT_PATH)
# 3. Tell the context not to check the hostname (essential for self-signed certs)
ssl_context.check_hostname = False
# 4. Set the verification mode
ssl_context.verify_mode = ssl.CERT_REQUIRED # verify every data packet sent by RMQ
via RMQ ID card and our certificate
# 5. Create Pika SSLOptions object using the context (avoids TypeError)
ssl_options = pika.SSLOptions(context=ssl_context)
# --- CONNECTION SETUP ---
credentials = pika.PlainCredentials(USERNAME, PASSWORD)
# this bundles everything a computer needs to know in order to connect to server
parameters = pika.ConnectionParameters(
    host=HOST,
    port=PORT,
    virtual_host=VHOST,
    credentials=credentials,
    ssl_options=ssl_options
)
try:
    print(f"Connecting securely to {HOST}:{PORT}...")
#This is the moment your code actually reaches out over the network to the RabbitMQ
server.
    connection = pika.BlockingConnection(parameters)
#opening a channel
    channel = connection.channel()
#checking if queue is present if not creates it
    channel.queue_declare(queue='DATA', durable=True)
```

```

print("Starting to publish 100 persistent messages via TLS...")
for i in range(1, 101):
    message_body = f"Secure Message Number {i}"
    channel.basic_publish(
        exchange='',
        routing_key='DATA',
        body=message_body,
        properties=pika.BasicProperties(
            delivery_mode=2,      # Make message persistent
        )
    )
    time.sleep(0.005)
print("\n--- Finished sending 100 secure messages ---")
connection.close()
except Exception as e:
    print(f"\n!!! TLS Publishing Error: {e}")
    print("Please check server status, firewall, and the CA_CERT_PATH.")

```

Step 4.9 : *vim consumer.py*

```

Python
import pika
import ssl

# --- CONFIGURATION ---
HOST = '192.168.64.56'
PORT = 5671
VHOST = '/data_processing'
USERNAME = 'publisher_user'
PASSWORD = 'RMQPassword@1101!'

# --- ABSOLUTE PATH CONFIRMED FROM YOUR HOST MACHINE ---
CA_CERT_PATH = '/Users/shambhavi.intern/ca_certificate.pem'

# --- TLS/SSL SETUP (The Safest Compatibility Fix) ---
# 1. Create the SSL Context, explicitly setting the protocol first (avoids
AttributeError)
ssl_context = ssl.SSLContext(ssl.PROTOCOL_TLSv1_2)

# 2. Load the CA file to verify the server's certificate
ssl_context.load_verify_locations(cafile=CA_CERT_PATH)

# 3. Tell the context not to check the hostname (essential for self-signed certs)
ssl_context.check_hostname = False

```

```

# 4. Set the verification mode
ssl_context.verify_mode = ssl.CERT_REQUIRED
# 5. Create Pika SSLOptions object using the context (avoids TypeError)
ssl_options = pika.SSLOptions(context=ssl_context)
# --- CONNECTION SETUP ---
credentials = pika.PlainCredentials(USERNAME, PASSWORD)
parameters = pika.ConnectionParameters(
    host=HOST,
    port=PORT,
    virtual_host=VHOST,
    credentials=credentials,
    ssl_options=ssl_options
)
def callback(ch, method, properties, body):
    print(f" [x] Received Secure Message: {body.decode()}") # runs everytime a
message arrives and decode it
try:
    print(f"Connecting securely to {HOST}:{PORT}...")
    connection = pika.BlockingConnection(parameters)
    channel = connection.channel()
    channel.queue_declare(queue='DATA', durable=True)
    print('\n [*] Waiting for secure messages on 5671. To exit press CTRL+C')
    channel.basic_consume(queue='DATA', on_message_callback=callback, auto_ack=True)
    channel.start_consuming()
except Exception as e:
    print(f"\n!!! TLS Consuming Error: {e}")
    print("Please check server status, firewall, and the CA_CERT_PATH.")

```

```
--- Finished sending 100 secure messages ---
shambhavi.intern@PP-R7MV533L02 ~ %
```

```
[*] Waiting for secure messages on 5671. To exit press CTRL+C
```

[illegible]

Connections

All connections (1)

Pagination

Page 1 of 1 - Filter: ☐ Regex ?

Displaying 1 item , page size up to: 100

Overview			Details			Network		+/-
Name	User name	State	SSL / TLS	Protocol	Channels	From client	To client	
192.168.64.1:64130	publisher_user	running	*	AMQP 0-9-1	1	0 B/s	0 B/s	

HTTP APIServer DocsTutorialsCommunity SupportCommunity SlackCommercial SupportPluginsGitHubChangelog